</> Ciência da Computação · Atividade de extensão

Do usuário ao criador

Guia completo do código do protótipo — como funciona, por que foi feito assim e como responder às perguntas do professor.

ESCOLHA → **CÓDIGO** → **RESULTADO**

Vite · React 19 · TypeScript · Tailwind CSS 4 · sem backend

Sumário

1. Visão geral do projeto
2. Tecnologias usadas e por quê
3. Como executar (e o que cada comando faz)
4. Estrutura de pastas e arquivos
5. Arquitetura: como os dados circulam
6. Conceitos de React que aparecem no código
7. Conceitos de TypeScript que aparecem no código
8. Arquivo por arquivo
    1. index.html e main.tsx
    2. data.ts — as opções e o estado
    3. code.ts — o gerador de código didático
    4. App.tsx — o "cérebro" da aplicação
    5. ControlPanel.tsx — o painel das etapas
    6. SitePreview.tsx — o site da turma
    7. CodePanel.tsx — o painel de código e o destaque
    8. BeforeAfter.tsx — a tela "Antes e Depois"
    9. index.css e site.css — os estilos
    10. Arquivos de configuração
9. Passo a passo: o que acontece quando...
10. Decisões de projeto (e as justificativas)
11. Perguntas prováveis do professor — com respostas
12. Limitações e melhorias possíveis
13. Glossário

1. Visão geral do projeto

O protótipo é uma **aplicação web de página única** (SPA – *Single Page Application*) usada como ferramenta de apresentação na oficina **"Do usuário ao criador"**. Ela tem duas áreas:

- **Painel de controle (esquerda)** — os apresentadores escolhem, em 6 etapas, o tema, o título, as cores, a fonte, o estilo do botão e a ação do botão.
- **Preview do site (direita)** — um "navegador falso" que mostra o site da turma. Ele começa propositalmente feio (parecendo HTML puro, sem CSS) e se transforma a cada escolha, **em tempo real**, sem botão de salvar.

Além disso, o preview tem o botão `</> Ver código`, que abre um painel com abas **HTML**, **CSS** e **JavaScript**. O código mostrado é gerado a partir das escolhas atuais e, quando algo muda, a linha correspondente **pisca** por cerca de 1 segundo. É isso que ensina a relação **ESCOLHA → CÓDIGO → RESULTADO**.

Por fim, há o **modo apresentação** (tela cheia, letras maiores, navegação por setas), a tela **Antes e Depois** ("Vocês fizeram isso.") e o botão **Recomeçar oficina**.

> **Resumo em uma frase (bom para começar uma resposta)**
>
> "É uma aplicação React em que existe **um único objeto de estado** com as escolhas da turma; o painel da esquerda **altera** esse estado, e o preview e o painel de código são **funções desse estado** — por isso tudo atualiza sozinho e ao mesmo tempo."

2. Tecnologias usadas e por quê

Tecnologia	Versão	Para que serve aqui
Node.js / npm	Node 18+	Ambiente que roda as ferramentas de desenvolvimento e instala as dependências (pacotes). O site final **não** precisa de Node para rodar; ele vira HTML/CSS/JS estático.
Vite	8	Servidor de desenvolvimento (recarrega a página instantaneamente ao salvar) e *bundler* que gera a versão final otimizada na pasta `dist/`.
React	19	Biblioteca para construir interfaces com **componentes**. A interface é descrita como função do estado: quando o estado muda, o React atualiza a tela.
TypeScript	7	JavaScript com **tipos**. Detecta erros antes de rodar (ex.: escrever `'roxa'` em vez de `'roxo'`). O navegador não entende TS: o Vite remove os tipos e entrega JS.
Tailwind CSS	4	CSS com classes utilitárias (`flex`, `p-6`, `border-ink`…) usado na interface da ferramenta.
@fontsource/*	5	Fontes (Inter, JetBrains Mono, Poppins, Playfair Display, Press Start 2P, Caveat) instaladas **dentro do projeto**, para funcionar **offline**, sem Google Fonts.

Não há backend, banco de dados, API, login ou serviço externo. Tudo roda no navegador. O único "armazenamento" é o `localStorage` do próprio navegador, usado só para não perder a criação se a página recarregar.

3. Como executar (e o que cada comando faz)

```
Terminal

npm install      # baixa as dependências listadas no package.json para node_modules/
npm run dev      # inicia o servidor do Vite em http://localhost:5173
npm run build    # verifica os tipos (tsc -b) e gera a versão final em dist/
npm run preview  # serve a pasta dist/ em http://localhost:4173
```

- `npm run dev` roda `vite --host`. O `--host` deixa o site acessível também por outros aparelhos na mesma rede (útil se o computador do projetor for outro).
- `npm run build` roda `tsc -b && vite build`: primeiro o **TypeScript confere os tipos** de todo o projeto; se houver erro, o build para. Depois o Vite junta e minifica tudo.
- Os scripts ficam no campo `"scripts"` do `package.json`.

4. Estrutura de pastas e arquivos

Árvore do projeto

```
dac/
├── index.html            ← página HTML base; só tem <div id="root"> e carrega main.tsx
├── package.json          ← nome do projeto, scripts e dependências
├── vite.config.ts        ← configuração do Vite (plugins React e Tailwind)
├── tsconfig.json         ← configuração do TypeScript
├── README.md             ← instruções curtas de uso
└── src/
    ├── main.tsx          ← ponto de entrada: importa fontes/CSS e monta o <App/>
    ├── App.tsx           ← componente principal: guarda o estado e organiza a tela
    ├── data.ts           ← tipos, opções (temas, cores, fontes…) e funções auxiliares
    ├── code.ts           ← gera o código HTML/CSS/JS "didático" a partir do estado
    ├── index.css         ← estilos da ferramenta (Tailwind + classes próprias)
    ├── site.css          ← estilos do "site da turma" e das animações dos temas
    └── components/
        ├── ControlPanel.tsx  ← painel da esquerda com as 6 etapas
        ├── SitePreview.tsx   ← o site da turma (preview)
        ├── CodePanel.tsx     ← painel com abas HTML/CSS/JS e destaque de linha
        └── BeforeAfter.tsx   ← tela "Antes → Depois"
```

A separação segue uma ideia simples: **dados** (`data.ts`), **lógica de geração de código** (`code.ts`), **componentes visuais** (`components/`) e **estilos** (`*.css`). O `App.tsx` conecta tudo.

5. Arquitetura: como os dados circulam

A regra mais importante do projeto: **existe uma única "fonte da verdade"**, o objeto `state` (do tipo `SiteState`), guardado no `App`.



1. O usuário clica numa opção no **ControlPanel**.
2. O ControlPanel chama a função `update(patch, key)` que recebeu do App por *props*. Ex.: `update({ color: 'roxo' }, 'color')`.
3. O `update` atualiza o estado com `setState` e registra qual coisa mudou (`flash`), para o destaque no código.
4. O React percebe que o estado mudou e **renderiza de novo** o App e seus filhos.
5. O **SitePreview** lê o novo estado e aplica a nova cor; o **CodePanel** gera de novo o código e faz piscar a linha marcada com `'color'`.

Esse padrão chama-se **fluxo de dados unidirecional** (*one-way data flow*): os dados descem por props (pai → filho) e as mudanças sobem por funções (filho chama função do pai). A vantagem é que **a tela nunca fica inconsistente**: o preview e o código são sempre calculados do mesmo estado.

6. Conceitos de React que aparecem no código

Componente

Uma função que devolve a interface (JSX). Ex.: `function SitePreview({ state }) { return <div>…</div> }`. Nome sempre com letra maiúscula.

JSX

Sintaxe parecida com HTML dentro do JavaScript. Diferenças: usa `className` em vez de `class`, `style` recebe um objeto (`style={{ color: "red" }}`) e expressões JS vão entre chaves `{ }`.

Props

Os "parâmetros" de um componente, passados pelo pai: `<SitePreview state={state} />`. O filho não altera props; se precisa mudar algo, chama uma função recebida (como `update`).

Estado — `useState`

```
const [state, setState] = useState<SiteState>(initial.state)
```

Cria uma variável que, quando alterada com `setState`, faz o React redesenhar o componente. **Nunca** se altera o estado diretamente (`state.color = …`); sempre se cria um objeto novo: `setState(s ⇒ ({ ...s, ...patch }))`. O `...` (spread) copia os campos antigos e sobrescreve os novos.

Efeitos — `useEffect`

Código que roda **depois** que a tela é desenhada, usado para coisas "fora do React": salvar no `localStorage`, ouvir o teclado, entrar em tela cheia, criar `setTimeout`. O segundo argumento (lista de dependências) diz **quando** rodar de novo. A função devolvida (`return () ⇒ …`) é a **limpeza**, que remove ouvintes e timers.

`useRef`

Guarda um valor que **não** causa nova renderização ao mudar. Usado para o contador `flashId` e para acessar um elemento do DOM (`areaRef` do painel de código).

`useCallback`

Memoriza uma função para ela não ser recriada a cada renderização. Usado em `setStep`.

Renderização condicional e listas

```
{codeOpen && <CodePanel ... />}          // só aparece se codeOpen for true
{theme ? <SiteComTema/> : <SiteCru/>}    // um ou outro
{THEMES.map((t) ⇒ <Card key={t.id} .../>)}  // lista; "key" identifica cada item
```

A `key` ajuda o React a saber qual item é qual entre renderizações. O projeto também usa um **truque** com `key`: **mudar a key de um elemento faz o React recriá-lo**, o que reinicia animações CSS (explicado no CodePanel).

`StrictMode`

Modo de desenvolvimento do React que ajuda a achar erros (por exemplo, roda efeitos duas vezes em desenvolvimento). Não afeta a versão final.

7. Conceitos de TypeScript que aparecem no código

União de literais (data.ts)

```
export type ColorId = 'none' | 'roxo' | 'azul' | 'verde' | 'laranja' | 'escuro'
```

Um **tipo união**: a variável só pode ter um desses textos. Se alguém escrever `'roxa'`, o TypeScript acusa erro no build.

Interface (data.ts)

```
export interface SiteState {
  theme: ThemeId
  title: string
  color: ColorId
  font: FontId
  button: ButtonId
  message: MessageId
  customMessage: string
}
```

Uma **interface** descreve o formato de um objeto. Outros utilitários usados:

- `Partial<SiteState>` — o mesmo objeto, mas com todos os campos opcionais. Usado no `update`, que recebe só o que mudou.
- `Exclude<ColorId, 'none'>` — o tipo sem a opção `'none'` (uma paleta real nunca tem id "none").
- `Omit<Palette, 'id' | 'label'>` — o tipo sem certos campos (usado na paleta padrão do navegador).
- `Record<Lang, ...>` — um objeto cujas chaves são exatamente `'html' | 'css' | 'js'`.
- `as const` — trava um array/objeto como constante, para o TS saber os valores exatos (usado em `STEPS`).
- `!` (non-null assertion) — "confie, isso não é nulo". Ex.: `document.getElementById('root')!`.

8. Arquivo por arquivo

8.1 `index.html` e `src/main.tsx`

O `index.html` é quase vazio: tem `<div id="root"></div>` e `<script type="module" src="/src/main.tsx">`. O ícone da aba é um SVG embutido (`data:` URL) com "</>".

src/main.tsx (resumido)

```
import '@fontsource/poppins/700.css'   // ...e as outras fontes (arquivos locais)
import './index.css'
import './site.css'
import App from './App'

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

O `main.tsx` é o **ponto de entrada**: importa as fontes e os CSS (o Vite transforma esses imports em arquivos na página) e diz ao React para desenhar o componente `<App />` dentro da div `root`.

8.2 `src/data.ts` — as opções e o estado

Arquivo **só de dados e funções puras** (sem interface). Se quiserem mudar um texto, uma cor ou uma mensagem, é aqui.

Estado inicial

```
export const INITIAL_STATE: SiteState = {
  theme: 'none',
  title: 'Meu site',
  color: 'none',
  font: 'none',
  button: 'none',
  message: 'none',
  customMessage: '',
}
```

Tudo começa em `'none'`, que significa "a turma ainda não escolheu" → o site aparece com o visual padrão do navegador. O botão **Recomeçar** volta exatamente para este objeto.

Listas de opções

- `THEMES` — 5 temas. Cada um tem: `emoji`, `eyebrow` (o selinho acima do título, ex. "Bola rolando"), `tagline` (parágrafo), `nav` (itens do menu), `deco` (qual arte desenhar: `'pixels' | 'equalizer' | 'field' | 'filmstrip' | 'stars'`), `cards` (3 cartões) e `titles` (sugestões de título).

- `PALETTES` — 5 paletas. Cada uma tem `bg` (fundo), `surface` (fundo dos cartões), `text`, `accent` (cor de destaque) e `onAccent` (cor do texto em cima do destaque).
- `DEFAULT_PALETTE` — branco/preto: simula o navegador sem CSS.
- `FONTS` — 4 fontes. `family` é usada no site de verdade; `css` é o texto que aparece no painel de código; `scale` corrige o tamanho (a Press Start 2P é muito larga → 0.62; a Caveat é pequena → 1.35).
- `BUTTONS` e `MESSAGES` — os 4 estilos de botão e as 3 mensagens prontas.

Funções auxiliares (helpers)

```
export const getTheme   = (s: SiteState) => THEMES.find((t) => t.id === s.theme)
export const getPalette = (s: SiteState) => PALETTES.find((p) => p.id === s.color)
export const getFont    = (s: SiteState) => FONTS.find((f) => f.id === s.font)
```

Transformam o **id** guardado no estado no **objeto completo**. Se o id for `'none'`, o `find` não acha nada e devolve `undefined` — e o resto do código usa isso para saber que "não há escolha" (por exemplo, `getPalette(state) ?? DEFAULT_PALETTE`; o `??` significa "se for nulo/undefined, use o da direita").

```
export function getMessage(s: SiteState): string | null {
  if (s.message === 'none') return null
  if (s.message === 'custom') return s.customMessage.trim() || '...'
  return MESSAGES.find((m) => m.id === s.message)!.text
}
```

Devolve o texto que o modal deve mostrar, ou `null` se ainda não há JavaScript (aí o botão do site não faz nada — de propósito).

buttonStyle (resumido)

```
export function buttonStyle(id, accent, onAccent, text): ButtonStyle | null {
  switch (id) {
    case 'arredondado': return { background: accent, color: onAccent, borderRadius: '999px',
border: 'none', boxShadow: 'none' }
    case 'quadrado':    return { ..., borderRadius: '0px', ... }
    case 'contorno':    return { background: 'transparent', color: accent, borderRadius:
'10px', border: `3px solid ${accent}`, ... }
    case 'sombra':      return { ..., border: `2px solid ${text}`, boxShadow: `6px 6px 0
${text}` }
    default:            return null
  }
}
```

Ponto importante: a mesma função é usada pelo **site** (para pintar o botão) e pelo **gerador de CSS** (para escrever as linhas `border-radius`, `box-shadow`...). Assim, o código mostrado e o resultado visual nunca divergem. Ela também é usada nos cards da etapa 05. O `999px` é um truque comum: um raio maior que o botão deixa as pontas totalmente redondas ("pílula").

```
export function slug(title: string): string {
  const s = title
    .normalize('NFD')                   // separa letras de acentos: "é" → "e" + "´"
    .replace(/[\u0300-\u036f]/g, '')    // remove os acentos
    .toLowerCase()
    .replace(/[^a-z0-9]+/g, '-')        // tudo que não é letra/número vira "-"
    .replace(/^-+|-+$/g, '')            // tira "-" do começo e do fim
  return s || 'meu-site'
}
```

Cria o endereço falso da barra do navegador: "Universo Gamer" → `universo-gamer.com.br`. Usa **expressões regulares** (regex).

8.3 `src/code.ts` — o gerador de código didático

> **Atenção: este é o ponto mais provável de pergunta**
>
> O código mostrado no painel **não é** o código-fonte real do projeto. É um código **didático e simplificado**, **gerado a partir do estado**, que representa de forma aproximada o que um site equivalente teria em HTML/CSS/JS puros. O site real é desenhado por componentes React (`SitePreview.tsx`). Isso foi uma decisão consciente: o público são estudantes que nunca programaram, e mostrar JSX/React confundiria a explicação HTML = estrutura, CSS = aparência, JS = comportamento.

A "linha de código" como dado

```
export type ChangeKey = 'theme' | 'title' | 'color' | 'font' | 'button' | 'message'

export interface CodeLine {
  text: string          // o texto da linha
  tags?: ChangeKey[]    // quais escolhas afetam esta linha
  comment?: boolean     // se é comentário (fica cinza/itálico)
}

const L = (text: string, ...tags: ChangeKey[]): CodeLine ⇒ ({ text, tags })
const C = (text: string): CodeLine ⇒ ({ text, comment: true })
```

Cada linha carrega **etiquetas** (`tags`) dizendo a qual escolha ela pertence. Ex.: `L(' background-color: #F3EEFF;', 'color')`. Quando a turma muda a cor, o painel faz piscar **todas as linhas com a tag `'color'`**. Esse é o mecanismo inteiro do destaque. `L` e `C` são apenas atalhos para criar linhas normais e comentários. O `...tags` (rest parameter) junta todos os argumentos extras num array.

`htmlCode(state)`

Sem tema escolhido, devolve só 3 linhas (o site "cru"):

HTML inicial gerado

```
<h1>Meu site</h1>
<p>Bem-vindo ao meu site.</p>
<button>Clique aqui</button>
```

Com tema, gera a estrutura completa: `<header>` com logo e `<nav>`, o selinho (`<p class="tema">`), o `<h1>`, o parágrafo, o botão, a **arte do tema** (tirada do objeto `ART_HTML`, uma lista de linhas para cada tipo de arte — ex. o campinho com `linha-do-meio`, `circulo-central`, `area` e `bola`) e a `<section class="cartoes">`. Quando uma mensagem é escolhida, o botão vira `<button onclick="explorar()">` — mostrando que o HTML "liga" o botão ao JavaScript.

`cssCode(state)`

Monta o CSS **só com o que foi escolhido**. Se nada foi escolhido, devolve o comentário "Ainda não tem CSS!". Senão, vai empilhando regras:

Exemplo de CSS gerado (com as tags indicadas)

```
body {
  background-color: #F3EEFF;   ← tag 'color'
  color: #1E1240;              ← tag 'color'
  font-family: "Poppins", sans-serif;   ← tag 'font'
}
h1 {
  color: #6C4DFF;              ← tag 'color'
}
button {
  background-color: #6C4DFF;   ← tags 'color' e 'button'
  border-radius: 999px;        ← tag 'button'
  ...
}
/* Arte do tema Esportes */
.campo, .area, .circulo-central {
  border: 3px solid #6C4DFF;   ← tags 'theme' e 'color'
}
.bola {
  animation: chutar 2.4s infinite;
}
```

A função `artCss(deco, accent)` produz as regras da arte de cada tema. No fim, um `while` remove linhas em branco sobrando no final.

`jsCode(state)`

```
function explorar() {
  mostrarMensagem("Você também pode criar tecnologia!");
}
```

Sem mensagem escolhida, mostra comentários explicando que "o botão existe (HTML) e tem visual (CSS), mas ainda não FAZ nada". `mostrarMensagem` é uma função **fictícia, didática**: no projeto real, quem abre o modal é o React (`setModal(msg)` no SitePreview).

Tabelas de ligação

```
export const CODE = { html: htmlCode, css: cssCode, js: jsCode }

export const LANG_OF = {
  theme: 'html', title: 'html',
  color: 'css', font: 'css', button: 'css',
  message: 'js',
}
```

`CODE` permite escolher o gerador pela aba atual (`CODE[tab](state)`). `LANG_OF` diz qual aba abrir automaticamente quando algo muda: mudou a cor → abre a aba CSS.

8.4 `src/App.tsx` — o "cérebro" da aplicação

Os estados

Estado	Para que serve
`state`	As escolhas da turma (`SiteState`).
`step`	Etapa atual (0 a 5).
`presenting`	Se o modo apresentação está ligado.
`codeOpen` / `codeTab`	Se o painel de código está aberto e qual aba.
`flash`	`{ keys, id }` : o que mudou por último (para piscar as linhas certas). O `id` sempre aumenta para reiniciar a animação.
`pulse`	Faz o botão "Ver código" piscar quando algo muda com o código fechado.
`transform`	Se a tela "Antes e Depois" está aberta.
`confirmReset`	Se a caixa de confirmação do "Recomeçar" está aberta.

Carregar e salvar no localStorage

```
const STORAGE_KEY = 'oficina-do-usuario-ao-criador'

function load() {
  try {
    const raw = localStorage.getItem(STORAGE_KEY)
    if (raw) {
      const parsed = JSON.parse(raw)
      return { state: { ...INITIAL_STATE, ...parsed.state }, step: Number(parsed.step) || 0 }
    }
  } catch { /* sem localStorage: começa do zero */ }
  return { state: INITIAL_STATE, step: 0 }
}

useEffect(() => {
  try { localStorage.setItem(STORAGE_KEY, JSON.stringify({ state, step })) } catch {}
}, [state, step])
```

- `localStorage` guarda **texto** no navegador, por site. Por isso usamos `JSON.stringify` (objeto → texto) e `JSON.parse` (texto → objeto).
- `{ ...INITIAL_STATE, ...parsed.state }`: se o dado salvo for de uma versão antiga e faltar algum campo, o valor inicial preenche.
- `try/catch`: em janela anônima ou com armazenamento bloqueado, o `localStorage` pode dar erro; o app continua funcionando.
- `useState(load)` passa a **função** (não o resultado), então o React só a chama uma vez, na primeira renderização.
- O `useEffect` com `[state, step]` salva toda vez que um dos dois muda.

A função `update` — o coração do projeto

```
const update = (patch: Partial<SiteState>, key: ChangeKey) => {
  setState((s) => ({ ...s, ...patch }))       // 1. aplica a mudança
  flashId.current += 1
  setFlash({ keys: [key], id: flashId.current }) // 2. marca o que mudou
  setCodeTab(LANG_OF[key])                    // 3. abre a aba certa
  if (!codeOpen) setPulse(flashId.current)    // 4. pisca "Ver código" se fechado
}
```

Toda escolha passa por aqui. `patch` é o pedaço que mudou e `key` diz **qual tipo** de mudança foi (usada para o destaque e para escolher a aba).

Navegação entre etapas

```
const setStep = useCallback(
  (n: number) => setStepRaw(Math.max(0, Math.min(STEPS.length - 1, n))), [])
```

"Prende" o número entre 0 e 5 (*clamp*): apertar → na última etapa não quebra nada.

Modo apresentação

```
useEffect(() => {
  document.documentElement.classList.toggle('presenting', presenting)
}, [presenting])

const enterPresentation = () => {
  setPresenting(true)
  document.documentElement.requestFullscreen?.().catch(() => {})
}
```

- Coloca a classe `presenting` no `<html>`. No `index.css`, `html.presenting { font-size: 19px }`. Como o Tailwind usa a unidade **rem** (relativa ao tamanho da fonte do `<html>`), **tudo aumenta proporcionalmente** com uma única linha.
- Pede **tela cheia** com a *Fullscreen API*. O `?.` (optional chaining) evita erro se o navegador não suportar; o `.catch` ignora se for negado.
- Um efeito ouve o evento `fullscreenchange`: se o usuário sair da tela cheia (o navegador usa o Esc para isso), o modo apresentação também desliga.
- No JSX, `presenting` troca o cabeçalho (mostra "1 / 6 Tema" com Anterior/Próximo), aumenta o painel esquerdo e diminui as margens do preview.

Atalhos de teclado

```
useEffect(() => {
  const onKey = (e: KeyboardEvent) => {
    if (e.key === 'Escape') {
      if (confirmReset) setConfirmReset(false)
      else if (transform) setTransform(false)
      else if (presenting) exitPresentation()
      return
    }
    if (!presenting || transform || confirmReset) return
    const t = e.target as HTMLElement
    if (t.tagName === 'INPUT' || t.tagName === 'TEXTAREA') return  // digitando? não navega
    if (e.key === 'ArrowRight') setStep(step + 1)
    if (e.key === 'ArrowLeft') setStep(step - 1)
  }
  window.addEventListener('keydown', onKey)
  return () => window.removeEventListener('keydown', onKey)
})
```

O **Esc** fecha primeiro o que está "por cima" (confirmação → antes/depois → modo apresentação). As setas só navegam no modo apresentação e **não** funcionam enquanto se digita (senão mover o cursor no campo de título trocaria de etapa). Este efeito não tem lista de dependências: ele é recriado a cada renderização para sempre enxergar os valores atuais de `step`, `presenting` etc.; a limpeza remove o ouvinte antigo, então nunca há dois ao mesmo tempo.

Recomeçar

`reset()` volta para `INITIAL_STATE`, etapa 0, fecha o código e limpa o destaque. Antes, abre uma caixa de confirmação própria (não o `window.confirm`, que é feio no projetor). Clicar fora da caixa ou

apertar Esc cancela.

O "navegador falso"

É só HTML/Tailwind: três bolinhas coloridas, a barra de endereço com `https://{slug}.com.br` e o botão `</> Ver código`. Abaixo vêm o `<SitePreview>` e, se aberto, o `<CodePanel>` ocupando 42% da altura.

8.5 `src/components/ControlPanel.tsx` — o painel das etapas

```
export const STEPS = [
  { n: '01', label: 'Tema',   lang: 'html', question: 'Sobre o que é o site da turma?' },
  { n: '02', label: 'Título', lang: 'html', question: 'Qual é o nome do site?' },
  { n: '03', label: 'Cores',  lang: 'css',  ... },
  { n: '04', label: 'Fonte',  lang: 'css',  ... },
  { n: '05', label: 'Botão',  lang: 'css',  ... },
  { n: '06', label: 'Ação',   lang: 'js',   ... },
] as const
```

A lista de etapas comanda as abas do topo, o número "01 / 06", a pergunta e o selo **HTML / CSS / JavaScript = ...** (vindo do objeto `CONCEPT`), que ajuda nas explicações da oficina.

O componente desenha a etapa atual com `{step === 0 && <ThemeStep/>}` etc. A `key={step}` na área de conteúdo faz a animação de entrada (`step-in`) tocar a cada troca de etapa.

O componente `Card`

```
<div role="button" tabIndex={0} onClick={onClick}
     onKeyDown={(e) => { if (e.target === e.currentTarget && (e.key === 'Enter' || e.key === '
')) { e.preventDefault(); onClick() } }}
     className={`choice ${selected ? 'selected' : ''}`}>
```

É uma `<div>` que **age como botão**. Por que não `<button>`? Porque o card da "Mensagem da turma" tem um `<input>` dentro, e HTML **não permite** input dentro de button. Para manter a **acessibilidade**, ela tem `role="button"` (leitores de tela a anunciam como botão), `tabIndex={0}` (dá para chegar com Tab) e responde a Enter/Espaço. O teste `e.target === e.currentTarget` evita que apertar espaço **dentro do input** acione o card.

As etapas

- **ThemeStep** — 5 cards; clicar chama `update({ theme: t.id }, 'theme')`.
- **TitleStep** — campo grande; `onFocus={e => e.target.select()}` seleciona tudo ao clicar (é só digitar por cima); botão × limpa; abaixo, a "mini-janela" mostra `<h1>título</h1>` ao vivo; e chips com sugestões do tema. `maxLength={40}` evita títulos gigantes.
- **ColorStep** — cada card pinta uma amostra com a própria paleta (fundo, bolinha de destaque, "Aa").
- **FontStep** — cada card escreve "Site da turma" na própria fonte, já com o `scale`.
- **ButtonStep** — cada card mostra um botão "Explorar" real, usando a mesma `buttonStyle()` do site e a cor de destaque escolhida.

- **ActionStep** — 3 mensagens + campo para mensagem própria (digitar já seleciona a opção "custom").

No rodapé: ← **Anterior**, contador e **Próximo** →; na última etapa, o "Próximo" vira **Ver transformação** ✨.

8.6 `src/components/SitePreview.tsx` — o site da turma

Variáveis CSS como "ponte" entre React e CSS

```
const vars = {
  '--bg': pal.bg,
  '--surface': pal.surface,
  '--text': pal.text,
  '--accent': pal.accent,
  '--on-accent': pal.onAccent,
  '--fs': font?.scale ?? 1,
  fontFamily: font?.family ?? DEFAULT_FONT,
} as CSSProperties

<div className="site-root" style={vars} data-theme={state.theme} data-font={state.font} data-
color={state.color}>
```

O componente não pinta cada elemento um por um. Ele define **variáveis CSS** (*custom properties*, as que começam com `--`) na div raiz, e o `site.css` usa `var(--accent)`, `var(--bg)`... em todo lugar. Trocar a cor = trocar 5 variáveis, e **tudo** (título, botão, campinho, planeta...) muda junto. É isso que mantém **cor, fonte e tema independentes**: a arte do tema usa a cor de destaque, seja ela qual for.

Os atributos `data-theme`, `data-font` e `data-color` permitem ao CSS fazer ajustes específicos, ex.: `.site-root[data-font='gamer'] h1 { line-height: 1.35 }`.

Dois "modos" de site

- **Sem tema** (`.site-raw`) — só `h1`, `p` e `button`, com margens e tamanhos copiados do padrão dos navegadores (fonte Times, h1 de `2em`, botão cinza de 13px). Assim parece HTML sem CSS.
- **Com tema** (`.site-themed`) — cabeçalho com menu, "hero" (título + arte) e cartões. A `key={theme.id}` faz o bloco ser recriado ao trocar de tema, tocando a animação de entrada `site-in`.

O modal (a parte "JavaScript")

```
const [modal, setModal] = useState<string | null>(null)

const onClick = () => {
  if (!interactive) return
  const msg = getMessage(state)
  if (msg) setModal(msg)       // sem mensagem escolhida → não faz nada
}

{modal && (
  <div className="site-modal-backdrop" onClick={() => setModal(null)}>
    <div className="site-modal" onClick={(e) => e.stopPropagation()}>
      <h2>{modal}</h2> ...
```

O pedido era não usar `window.alert()`; então o modal é um elemento do próprio site, posicionado com `position: absolute` por cima do preview (só dentro do "navegador", não da tela toda). Clicar no fundo escuro fecha; `e.stopPropagation()` impede que o clique **dentro** da caixa "suba" até o fundo e feche sem querer. A prop `interactive=false` é usada na tela Antes/Depois, onde o site é só uma imagem.

A arte de cada tema (`DecoArt`)

Tudo feito com **divs + CSS**, sem imagens externas (funciona offline):

- **Games** — o invasor é uma **matriz de texto**: cada `X` vira um quadradinho colorido num grid de 11 colunas.
- **Música** — 14 barras com `animation-delay` e duração diferentes (calculados com `%`), para parecer um equalizador "aleatório" mas determinístico.
- **Esportes** — linha do meio, círculo central e duas áreas são divs com borda; a bola ⚽ faz a animação `kick`.
- **Filmes** — uma fita girada −8°; os furinhos são `radial-gradient` repetidos; a fita desliza com `roll`.
- **Espaço** — 28 estrelas posicionadas com `(i * 37) % 100`%, piscando; o planeta é um círculo com gradiente e o anel é um `::after`.

```
const INVADER = [
  '..X.....X..',
  '...X...X...',
  '..XXXXXXX..',
  '.XX.XXX.XX.',
  'XXXXXXXXXXX',
  'X.XXXXXXX.X',
  'X.X.....X.X',
  '...XX.XX...',
]
// cada caractere vira um <span>; os "X" recebem a classe "on" (pintada com var(--accent))
```

8.7 `src/components/CodePanel.tsx` — o painel de código e o destaque

```
const lines = CODE[tab](state)

{lines.map((line, i) => {
  const hit = !!flash && !!line.tags?.some((k) => flash.keys.includes(k))
  return (
    <div key={hit ? `${i}-${flash!.id}` : i} className={`code-line ${hit ? 'flash' : ''}`}>
      <span className="code-num">{i + 1}</span>
      ...
```

Como a linha pisca:

1. Para cada linha, verifica se alguma de suas `tags` está em `flash.keys` (o que acabou de mudar).
2. Se sim, recebe a classe `flash`, que tem uma animação CSS de 1,4s (fundo roxo que some).
3. **O truque da key:** uma animação CSS só toca quando a classe é aplicada pela primeira vez. Se a turma clicar em Roxo e depois em Azul, a linha já tem a classe e não tocaria de novo. Por isso a `key` das linhas destacadas inclui `flash.id`, que muda a cada alteração: o React **recria** o elemento e a animação recomeça do zero.

```
Rolagem automática até a linha destacada

useEffect(() => {
  areaRef.current?.querySelector('.flash')?.scrollIntoView({ block: 'nearest', behavior:
'smooth' })
}, [flash, tab])
```

Realce de sintaxe (cores do código)

A função `highlight(text, lang)` é um realce **bem simples**, feito com uma regex por linguagem. Ela percorre a linha com `re.exec` e envolve os pedaços encontrados em `<span className="tok-...">`: tags HTML em rosa, atributos/propriedades em azul, textos entre aspas em verde, cores `#hex` em amarelo, palavra-chave `function` em lilás. Comentários são marcados no gerador (`comment: true`) e ficam em cinza itálico. Não é um analisador de verdade (não precisa ser): o código gerado é previsível e curto. Usar uma biblioteca (Prism, Shiki) seria exagero para a oficina.

8.8 `src/components/BeforeAfter.tsx` — a tela "Antes e Depois"

```
useEffect(() => {
  setPhase(0)
  const t1 = setTimeout(() => setPhase(1), 1600)   // DEPOIS entra
  const t2 = setTimeout(() => setPhase(2), 2700)   // "Vocês fizeram isso." + confete
  return () => { clearTimeout(t1); clearTimeout(t2) }
}, [run])
```

- Uma pequena **máquina de estados** com 3 fases: 0 = só ANTES, ampliado; 1 = DEPOIS entra pela direita com efeito de "mola" (`cubic-bezier`); 2 = frase e confete.

- As transições usam classes Tailwind (`transition-all duration-700`, `opacity-0` → `opacity-100`...) trocadas conforme a fase.
- "↺ Rever" incrementa `run`, o que roda o efeito de novo. A limpeza cancela timers pendentes (se fechar no meio, nada "vaza").
- **ANTES** é literalmente `<SitePreview state={INITIAL_STATE}/>` e **DEPOIS** é `<SitePreview state={state}/>`: o mesmo componente, com estados diferentes — uma boa demonstração de reuso de componentes.
- **Miniatura:** o site é desenhado em 1000×640 px e reduzido com `transform: scale(k)`. O `k` é calculado pelo tamanho da janela (`useScale`) e recalculado no evento `resize`.
- **Confete:** 60 `<span>` coloridos com a animação CSS `fall`, com atrasos e durações diferentes.

8.9 `src/index.css` e `src/site.css` — os estilos

index.css (a ferramenta)

```
@import 'tailwindcss';

@theme {
  --color-paper: #f4f4f3;   /* cria as classes bg-paper, text-paper... */
  --color-ink: #111111;     /* border-ink, text-ink... */
  --color-accent: #6c4dff;  /* bg-accent, text-accent... */
  --font-mono: 'JetBrains Mono', ui-monospace, monospace;
}
```

No **Tailwind 4**, a configuração fica no próprio CSS: cada variável em `@theme` gera classes utilitárias. Em `@layer components` ficam classes próprias reutilizáveis: `.btn-ink`, `.btn-accent`, `.btn-ghost`, `.chip`, `.choice` (os cards — com "sombra dura" deslocada ao passar o mouse, estilo editorial) e `.choice.selected`. Também estão aqui as animações `step-in`, `pulse`, `code-flash`, as cores do realce (`.tok-*`) e o confete.

site.css (o site da turma)

- `.site-root` usa `font-size: calc(16px * var(--fs))`: o tamanho de tudo no site é em `em`, então ajustar `--fs` por fonte reescala o site inteiro.
- `.site-root * { transition: background-color 0.5s, color 0.5s, border-radius 0.4s, ... }`: é isso que faz as mudanças de cor e de botão serem **suaves** em vez de instantâneas.
- `color-mix(in srgb, var(--text) 12%, transparent)`: mistura cores no próprio CSS (bordas e fundos "fracos" que funcionam em qualquer paleta, inclusive no modo escuro).
- Cada tema tem uma "atmosfera" no fundo do hero com `background-image`: grade quadriculada (Games), pontinhos (Espaço), ondas (Música), faixas de gramado (Esportes), brilho (Filmes).
- `@keyframes` definem as animações: `bob`, `slide`, `eq`, `kick`, `roll`, `twinkle`, `fly`, `pop`...
- O Tailwind "zera" estilos padrão dos elementos (*preflight*). Por isso o `site.css` recria à mão o visual padrão do navegador para o site "cru".

8.10 Arquivos de configuração

vite.config.ts

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [react(), tailwindcss()],
})
```

- **vite.config.ts** — liga dois plugins: o do React (entende JSX/TSX) e o do Tailwind (gera o CSS das classes usadas).
- **tsconfig.json** — `"strict": true` (verificações rigorosas), `"noEmit": true` (o TS só verifica; quem gera JS é o Vite), `"jsx": "react-jsx"`, `"noUnusedLocals"` / `"noUnusedParameters"` (acusa variáveis não usadas).
- **package.json** — `dependencies` (vão para o site: React e fontes) e `devDependencies` (só para desenvolver: Vite, TypeScript, Tailwind, tipos). `"type": "module"` usa `import/export` modernos.
- **.gitignore** — ignora `node_modules` e `dist`.

9. Passo a passo: o que acontece quando...

...clico no card "Roxo"?

1. `ColorStep` → `Card.onClick` → `update({ color: 'roxo' }, 'color')`.
2. `setState`: `state.color` passa a ser `'roxo'`. `setFlash({ keys: ['color'], id: 7 })`. `setCodeTab('css')`.
3. React renderiza de novo. O `SitePreview` faz `getPalette(state)` → objeto Roxo → define `--bg: #F3EEFF`, `--accent: #6C4DFF`...
4. O CSS reage às variáveis; as `transition` fazem a mudança em 0,5s.
5. O `CodePanel` chama `cssCode(state)`: agora existem as linhas de `background-color`, `color` do body e do h1, todas com a tag `'color'` → recebem a classe `flash` com `key` nova → piscam.
6. O `useEffect` do App salva o novo estado no `localStorage`.

...digito no campo de título?

Cada tecla dispara `onChange` → `update({ title: valor }, 'title')`. O `<h1>` do site, o texto do logo, a barra de endereço (`slug`) e a linha `<h1>` do código mudam a cada letra. Se o campo ficar vazio, `displayTitle` devolve "Meu site".

...clico no botão "Explorar" do site?

`onClick` do SitePreview → `getMessage(state)`. Se a etapa 06 ainda não foi feita, devolve `null` e nada acontece (o momento pedagógico: "sem JavaScript, o botão não faz nada"). Se houver mensagem, `setModal(msg)` e o modal aparece com animação `pop`.

...aperto → no modo apresentação?

O ouvinte `keydown` do App verifica: está no modo apresentação? não há modal aberto? não estou digitando? Então `setStep(step + 1)`, limitado a 5. O ControlPanel mostra a nova etapa com a animação `step-in`.

...recarrego a página (F5)?

`useState(load)` lê o `localStorage` e restaura as escolhas e a etapa. Modo apresentação e painel de código voltam fechados (não são salvos, de propósito).

10. Decisões de projeto (e as justificativas)

Decisão	Por quê
Um único estado no App	Preview e código sempre coerentes; fácil de salvar, restaurar e resetar (é um objeto só).
Código do painel é "didático", não o real	O público nunca programou; HTML/CSS/JS puros explicam melhor do que JSX/React. O código é gerado do mesmo estado, então reflete as escolhas de verdade.
Linhas com "tags"	Forma simples de saber o que destacar, sem comparar textos antigos e novos (diff).
Variáveis CSS para as cores	Uma troca de cor atualiza tudo; mantém tema, cor e fonte independentes.
Arte dos temas só com CSS/emoji	Funciona offline, não depende de imagens externas, e a arte acompanha a cor escolhida.
Fontes via @fontsource	Ficam dentro do projeto → funcionam sem internet no dia da oficina.
Sem backend/banco	Requisito: simples de executar hoje, sem login, sem API. Nada precisa ser compartilhado entre computadores.
localStorage	Evita perder a criação se a página recarregar; `try/catch` garante que o app funcione mesmo sem ele.
Modal próprio em vez de alert()	Requisito; alert é feio, bloqueia a página e não segue o visual do site.
Escala por rem no modo apresentação	Uma linha de CSS aumenta toda a interface de forma proporcional para o projetor.
Tailwind na ferramenta, CSS próprio no site	Tailwind é ótimo para layout fixo; o site precisa de estilos dinâmicos (variáveis, keyframes), mais claros em CSS comum.

11. Perguntas prováveis do professor — com respostas

P: O código que aparece no painel "Ver código" é o código real do site?

R: Não exatamente. É um código **didático gerado a partir do estado** (em `code.ts`). Ele mostra o que um site equivalente teria em HTML/CSS/JS puros. O preview real é feito com componentes React. Escolhemos assim porque o objetivo é ensinar o que cada linguagem faz a quem nunca programou. Os valores (cores, fontes, raio do botão, mensagem) são **exatamente** os usados no preview, porque vêm das mesmas funções (`getPalette`, `buttonStyle`...).

P: Como vocês sabem qual linha destacar?

R: Cada linha gerada tem uma lista de `tags` (`'color'`, `'font'`, `'button'`...). Quando algo muda, a função `update` registra qual tag mudou no estado `flash`. O CodePanel aplica a classe `flash` nas linhas que têm essa tag. Para a animação tocar de novo a cada mudança, a `key` da linha inclui um contador, forçando o React a recriar o elemento.

P: Por que usar React e não só HTML/JS?

R: Porque a tela inteira depende de um conjunto de escolhas. No React descrevemos a interface como **função do estado**; quando o estado muda, ele atualiza só o que mudou no DOM. Com JS puro teríamos que atualizar manualmente o preview, o código, a URL, os cards selecionados... e seria fácil esquecer algum e deixar a tela inconsistente.

P: O que é estado? E props?

R: **Estado** são os dados que mudam com o tempo e, quando mudam, fazem o componente redesenhar (`useState`). **Props** são os dados que um componente pai passa ao filho, como parâmetros de função. Aqui o estado fica no `App` e desce por props para o painel, o preview e o código.

P: Por que o estado fica no App e não em cada componente?

R: Porque três componentes precisam dele (ControlPanel, SitePreview, CodePanel). Em React, quando irmãos compartilham dados, o estado sobe para o ancestral comum (*lifting state up*).

P: O que é o useEffect e onde vocês usam?

R: É para efeitos colaterais, coisas fora da renderização: salvar no localStorage, ouvir teclado, ouvir `fullscreenchange`, ouvir `resize`, timers da tela Antes/Depois e rolar até a linha destacada. A função de retorno faz a limpeza (remove ouvintes, cancela timers).

P: Por que não alterar o estado diretamente, tipo `state.color = "roxo"`?

R: O React compara a **referência** do objeto para saber se algo mudou. Alterando o mesmo objeto, ele não percebe e não redesenha. Por isso criamos um objeto novo com `{ ...s, ...patch }` (imutabilidade).

P: Onde os dados ficam salvos? Tem banco de dados?

R: Não há banco nem servidor. O estado é salvo no `localStorage` do navegador, em JSON, na chave `oficina-do-usuario-ao-criador`. Só serve para não perder a criação se a página recarregar. Fica só naquele navegador, naquele computador.

P: Funciona sem internet?

R: Sim, depois do `npm install`. As fontes estão no projeto (@fontsource), não há imagens externas (a arte é CSS e emoji) e não há chamadas de API.

P: Como funciona a mudança de cor em todo o site de uma vez?

R: O SitePreview define variáveis CSS (`--bg`, `--text`, `--accent`...) no elemento raiz. Todo o `site.css` usa `var(--accent)` etc. Trocar a paleta muda as variáveis e tudo herda. As `transition` deixam a troca suave.

P: Como o modo apresentação aumenta tudo?

R: Coloca a classe `presenting` no `<html>`, que muda o `font-size` raiz de 16px para 19px. Como o Tailwind usa `rem`, tudo escala junto. Também pede tela cheia (Fullscreen API) e troca o cabeçalho por um com o indicador "1 / 6" e os botões Anterior/Próximo.

P: Por que as setas não trocam de etapa quando estou digitando o título?

R: O ouvinte de teclado verifica `e.target.tagName`: se for `INPUT` ou `TEXTAREA`, ignora. Senão, mover o cursor no texto trocaria de etapa.

P: O que é TypeScript e qual a vantagem aqui?

R: É JavaScript com tipos. Por exemplo, `ColorId` só aceita `'none' | 'roxo' | …`; um erro de digitação vira erro de compilação. O `npm run build` roda `tsc -b` antes, então código com erro de tipo nem chega a gerar build.

P: O que o Vite faz?

R: Em desenvolvimento, serve os arquivos e atualiza o navegador na hora ao salvar (HMR). No build, transforma TSX em JS, junta os módulos, processa o Tailwind, copia as fontes e minifica tudo em `dist/`, que pode ser hospedado em qualquer servidor estático.

P: Por que o card de opção é uma div com role="button" e não um button?

R: Porque o card da mensagem personalizada contém um `<input>`, e HTML não permite input dentro de button. Para manter acessibilidade: `role="button"`, `tabIndex={0}` e tratamento de Enter/Espaço.

P: Como foi feito o invasor do Space Invaders?

R: Com uma matriz de strings (`INVADER`): cada `X` vira um quadradinho pintado com a cor de destaque num CSS Grid de 11 colunas. Não é imagem.

P: Como funciona a tela "Antes e Depois"?

R: Ela reutiliza o componente `SitePreview` duas vezes: com `INITIAL_STATE` (antes) e com o estado atual (depois), reduzidos com `transform: scale`. Dois `setTimeout` avançam as fases da animação; a limpeza do `useEffect` cancela os timers.

P: E se o professor perguntar o que é `mostrarMensagem`?

R: É uma função **fictícia** no código didático, com nome em português para ficar compreensível. Representa "abrir uma janelinha com a mensagem". No projeto real, isso é `setModal(msg)` no SitePreview, que faz o React desenhar o modal.

P: O que acontece se o título ficar vazio?

R: `displayTitle` devolve "Meu site" e `slug` devolve "meu-site", então o site nunca fica sem título nem com URL quebrada.

P: Como o projeto foi testado?

R: Com `npm run build` (verificação de tipos + build sem erros) e um teste no Chromium em modo headless, automatizado com Puppeteer: clica em todas as etapas, digita o título, abre o modal, a tela Antes/Depois, o modo apresentação (setas e Esc), troca os 5 temas e o modo escuro, faz o reset e confere que não há erros no console.

12. Limitações e melhorias possíveis

- **Código didático ≠ código real**: é uma aproximação. Uma melhoria seria um botão "baixar site" que gerasse um arquivo HTML de verdade com essas escolhas, para os alunos abrirem em casa.
- **Sem testes automatizados no repositório**: os testes foram feitos manualmente/por script. Poderiam ser adicionados com Vitest (funções de `data.ts` e `code.ts` são puras e fáceis de testar).
- **Realce de sintaxe simplificado**: baseado em regex; funciona para o código gerado, mas não para código arbitrário.
- **Um computador só**: não há votação dos alunos pelo celular. Isso exigiria um servidor (WebSocket), o que fugia do requisito "sem backend".
- **Layout pensado para telas largas** (notebook/projetor), não para celular.
- **Fontes**: os títulos ficam bem até ~40 caracteres (limite do campo).

13. Glossário

SPA	Single Page Application: um site que carrega uma página só e atualiza o conteúdo com JavaScript.
Componente	Função React que devolve um pedaço da interface.
JSX / TSX	Sintaxe parecida com HTML dentro de JS/TS. TSX = JSX com TypeScript.
Estado (state)	Dados que mudam e fazem a tela ser redesenhada.
Props	Dados passados de um componente pai para um filho.
Hook	Funções do React que começam com "use" (useState, useEffect, useRef, useCallback).
DOM	A árvore de elementos da página que o navegador mostra.
Renderizar	Calcular e desenhar a interface a partir do estado.
Bundler	Ferramenta que junta e otimiza os arquivos do projeto (aqui, o Vite).
Variável CSS	Valor reutilizável no CSS, ex.: --accent, lido com var(--accent).
rem / em	Unidades relativas: rem ao tamanho de fonte do <html>; em ao do elemento pai.
Keyframes	Definição de uma animação CSS por etapas.
localStorage	Armazenamento de texto do navegador, por site, que sobrevive a recarregar a página.
JSON	Formato de texto para representar objetos: {"color":"roxo"}.
Regex	Expressão regular: padrão para buscar/substituir texto.
Slug	Versão de um texto própria para URL: "Universo Gamer" → universo-gamer.
Acessibilidade	Fazer a interface funcionar para todos, inclusive com teclado e leitores de tela.
Headless	Navegador rodando sem janela, usado para testes automatizados.